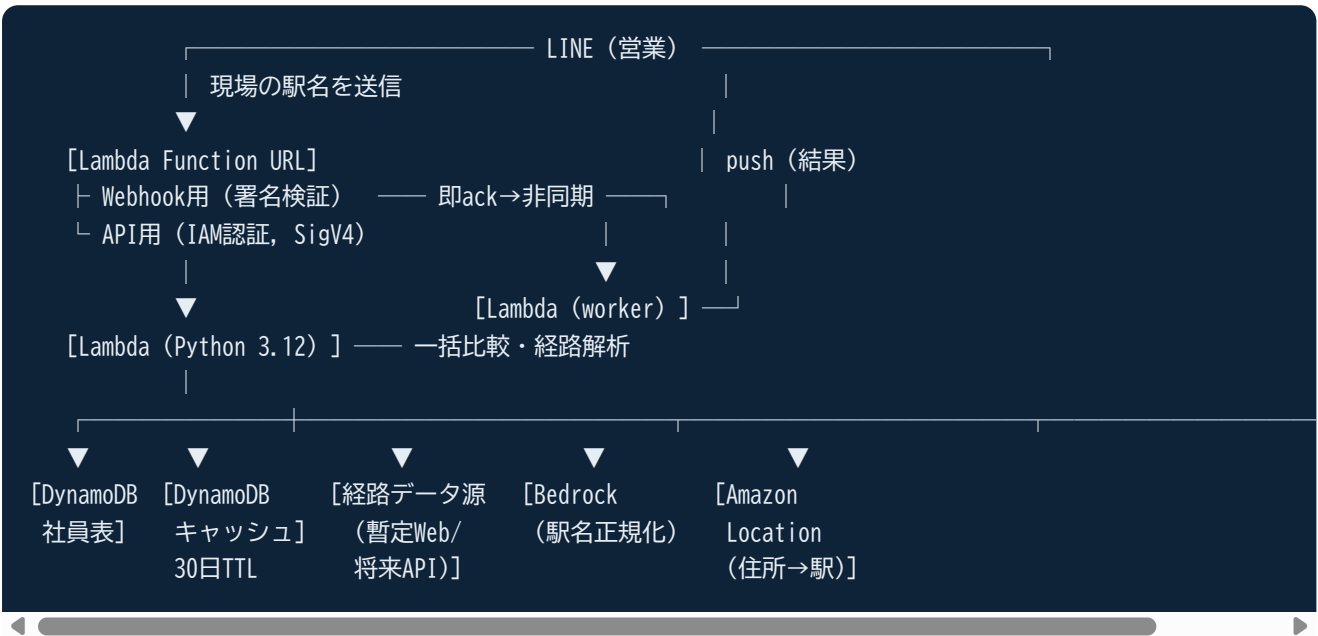


要員通勤コスト比較ツール アーキテクチャ設計書

本書は、本ツールの全体構成・データフロー・設計方針をまとめたものです。

1. 全体構成（AWS サーバーレス）

東京リージョン（ap-northeast-1）。AWS CDK（TypeScript）で IaC 化しています。



2. 主要コンポーネント

コンポーネント	役割
Lambda (API)	<code>/query</code> (単一)・ <code>/site</code> (一括比較)・ <code>/staff</code> ・ <code>/nearest</code> を提供。Function URL は IAM 認証
Lambda (LINE Webhook)	LINE 署名を検証 → 即「収集中…」ack → 自分自身を非同期起動 → 結果を push
DynamoDB 社員表	PK=社員ID (花名冊と同一キー)。最寄駅・状態を保持 (住所は持たない)
DynamoDB キャッシュ	PK=区間キー。結果を 30 日 TTL で保持し再取得を抑制
経路データ源	抽象インターフェースの実装 (暫定 Web 解析 / 将来 認可 API)
Bedrock	中国語等の駅名表記を日本語駅名へ正規化 (兜底。通常は未使用)
Amazon Location	住所のジオコーディング→最寄駅判定

3. データフロー（LINE 一括比較）

1. LINE → Webhook Lambda。X-Line-Signature を channel secret で検証。
2. 即「収集中…」を返信し、重い処理は **非同期 worker** へ委譲、Webhook は即 200 を返す（タイムアウト回避）。
3. worker：現場駅を正規化（かな/ローマ字/中国語）→ DynamoDB から待機中要員を取得。
4. 各要員の最寄駅→現場をデータ源に問い合わせ（同一区間はキャッシュ命中で再取得しない、リクエスト間にスリープ）。
5. 戦略（定期最安／時間最短）で 1 経路ずつ選定し、結果を **push** で返信。

4. 設計方針

4.1 差し替え契約（最重要）

データ取得元を抽象インターフェース（query(出発, 到着, 戦略) → 共通結果型）の背後に隠蔽。業務層・社員 DB は具象実装を知りません。新データ源はインターフェース実装のみで追加でき、暫定 Web から正式 API への移行も業務ロジック無改修で行えます。

4.2 責任あるデータ取得

リクエストは直列・低頻度・間隔スリープ。結果は 30 日キャッシュし、同一区間は再取得しません。小規模・低トラフィック前提の設計です。

4.3 個人情報配慮

社員は住所を保存せず最寄駅のみ保持。LINE 認証情報（secret/token）は **SSM SecureString** に格納し、コード・環境変数に値を置きません。

4.4 非同期化

LINE の応答制限を超えないよう「即 200 + push」方式を採用。検索が重くてもタイムアウトしません。

5. 技術選定の理由

選定	理由
サーバーレス（Lambda/DynamoDB）	低トラフィックで常時起動コスト不要・従量課金
Function URL	API Gateway を介さず軽量に HTTP エンドポイントを提供
AWS_IAM 認証（API）	公開 URL からのスクレイピング誘発を防ぐ
Bedrock	中国語等の曖昧入力を堅牢に正規化（ルールベースより頑健）

選定	理由
Amazon Location	住所→座標→最寄駅をマネージドに実現
CDK	構成をコード化し再現性のあるデプロイ